

SISTEMAS OPERACIONAIS

```
mkdir "Universidade Federal de Alfenas"  
touch "Universidade Federal de Alfenas/Turma SO 2025-02.txt"  
echo "Professor Romário Borges" > "Universidade Federal de Alfenas/Turma SO  
2025-02.txt"
```

Comunicação Inter Processo (IPC)

```
cat aula03.txt
```

Na aula de Hoje, teremos:

- Entender como acontece a comunicação entre processos;
- Como é realizado a comunicação entre processos;
- Desafios como: condição de corrida, áreas críticas e exclusão mútua;
- Entender algumas soluções como a Espera ocupada, Algoritmo de Peterson, Dormir e Acordar;
- O problema do Produtor-consumidor, semáforos e monitores;

Objetivo da IPC

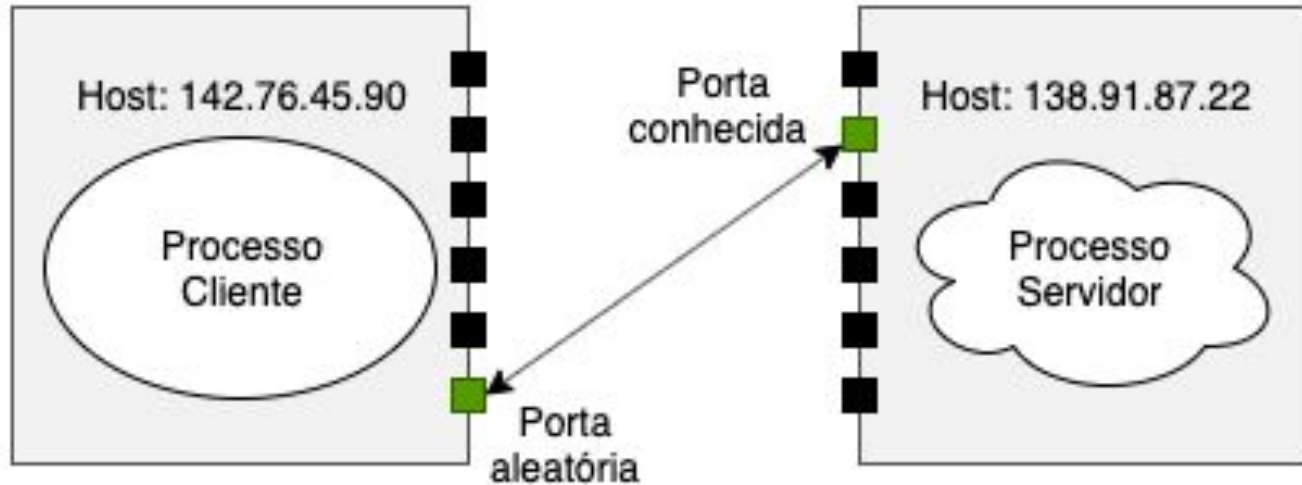
- Atender a usuários simultâneos;
- Aplicações complexas;
- Máquinas com múltiplos processadores;

Como é realizado a comunicação?

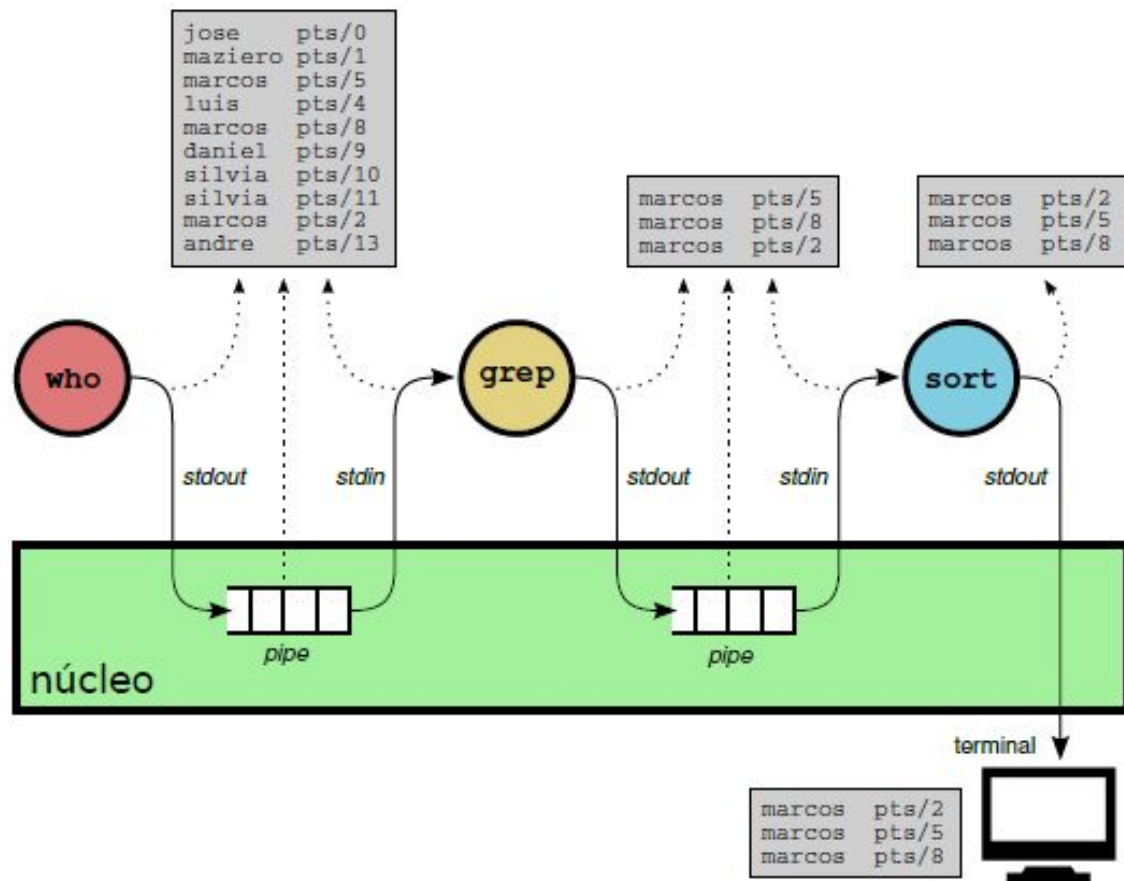
- Pipes (em sistemas Unix);
- Sockets (Redes);
- **Memória compartilhada;**
- Entre outras formas;

Sockets

Redes: Comunicação baseada em endereços e portas



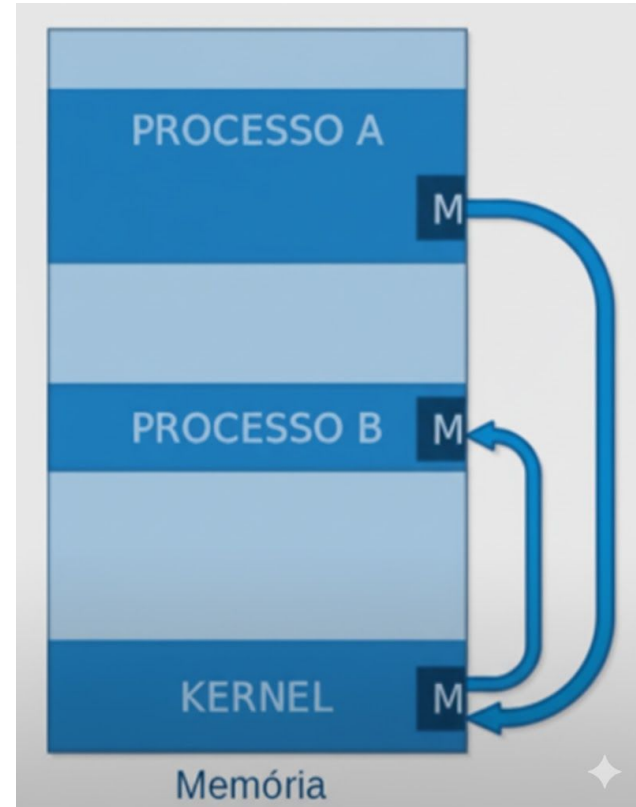
Pipes



Recordando...

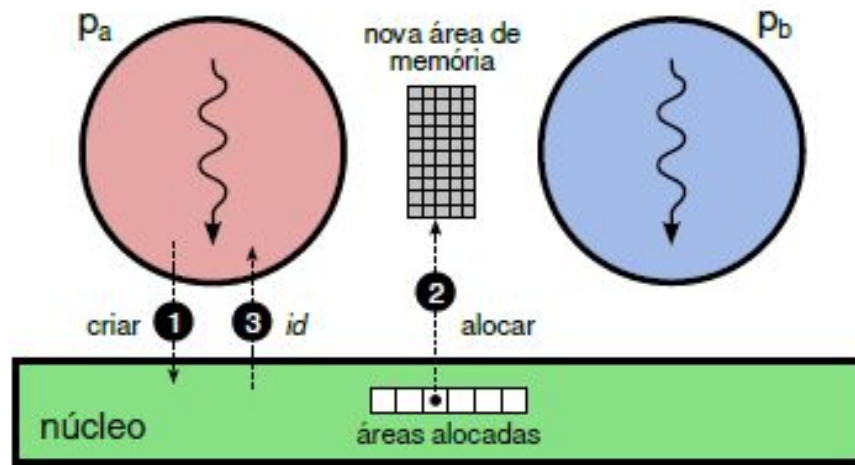
Chamadas de Sistema:

Acontece a troca de contexto da CPU: troca de modo usuário para modo núcleo.

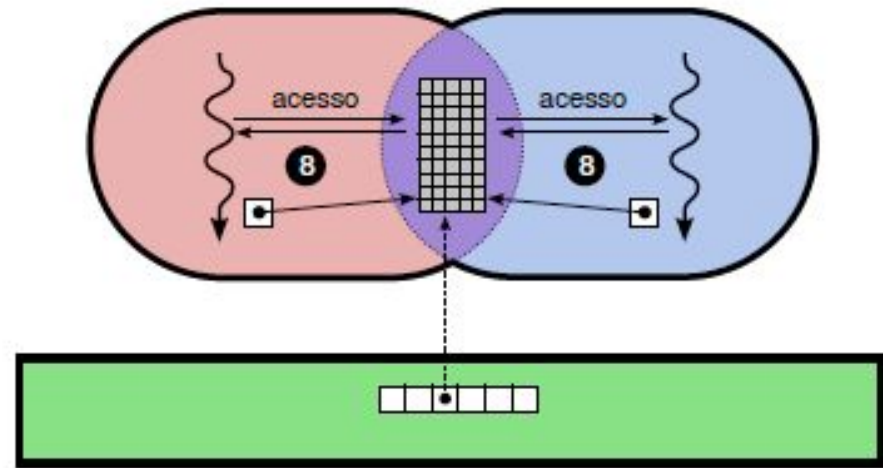
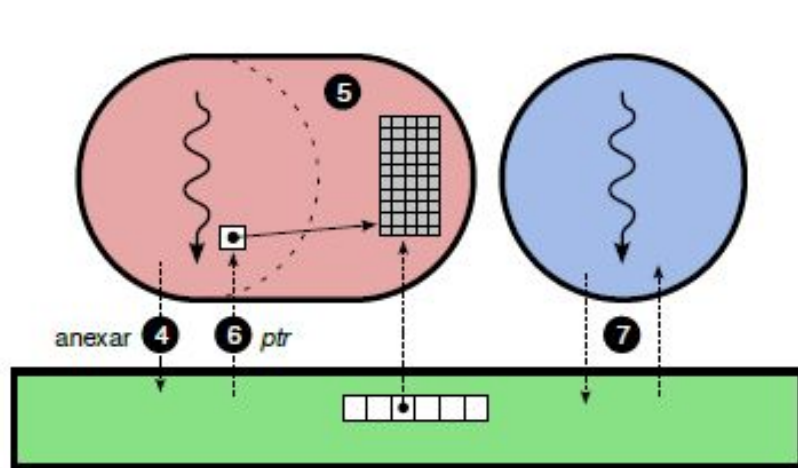


Memória compartilhada

O SO gerencia as áreas e os processos, mas o acesso é feito pelo próprio processo.



Memória compartilhada



Desafio: Condição de corrida

Condição de corrida:

Quando dois ou mais processos precisam acessar um recurso, e a ordem de quem acessa primeiro, é muito importante!

Thread 1	Thread 2		Inteiro
			0
lê o valor		←	0
	lê o valor	←	0
incrementa-o			0
	incrementa-o		0
escreve nele		→	1
	escreve nele	→	1

Exemplo: Condição de corrida

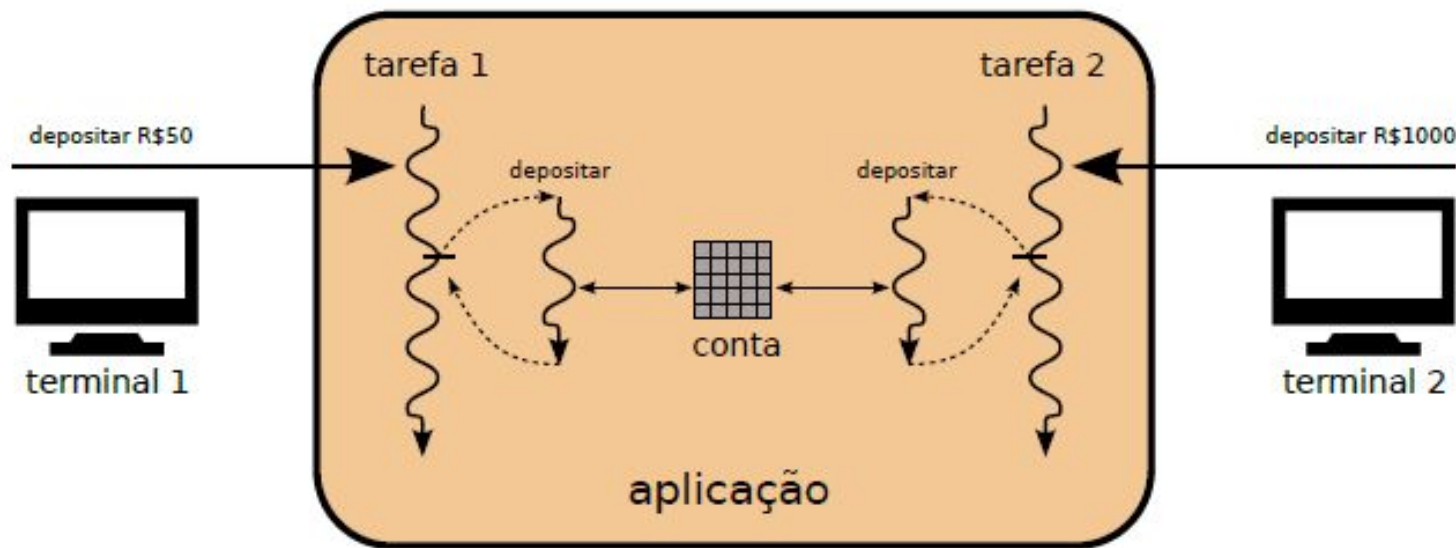
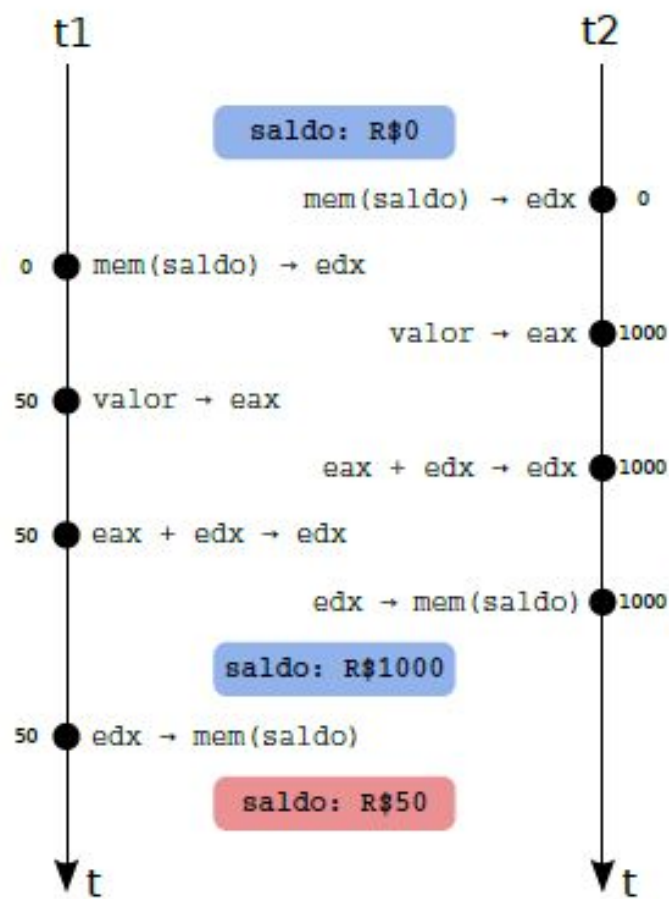
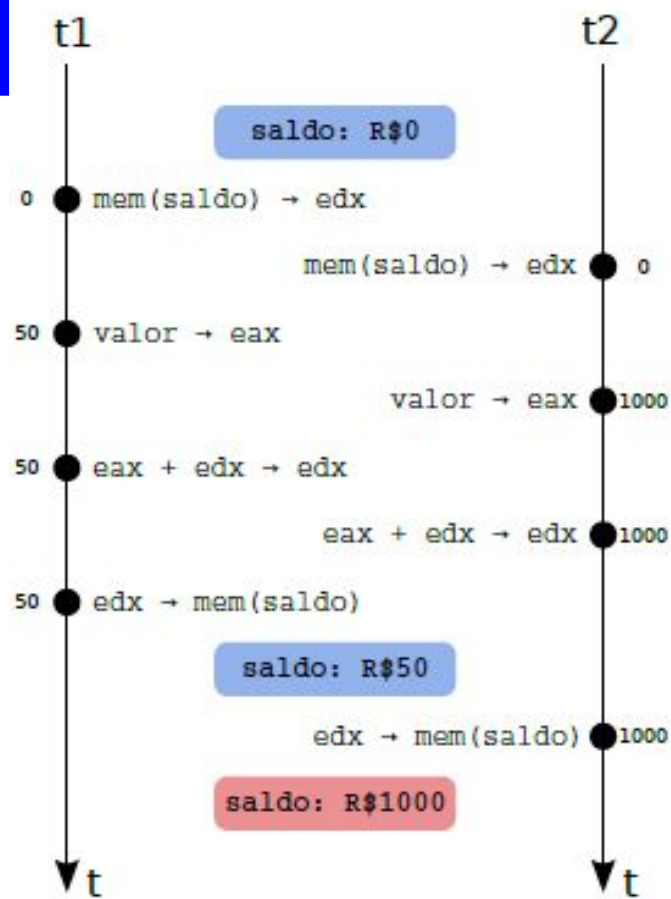
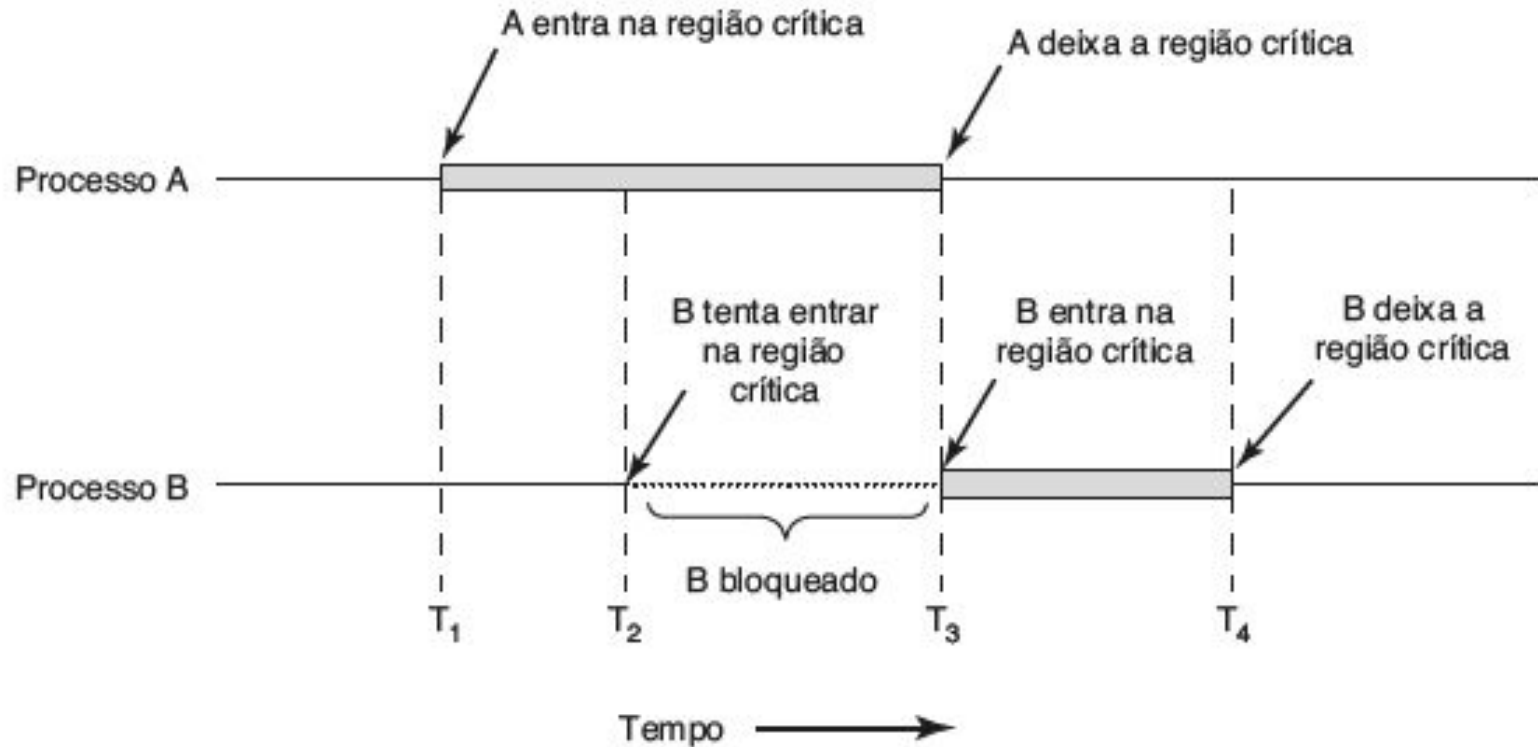


Figura 10.1: Acessos concorrentes a variáveis compartilhadas.



Áreas críticas e Exclusão mútua



Mecanismos para evitar para Exclusão mútua

1. 2 processos não podem ocupar a região crítica ao mesmo tempo;
2. Independência de fatores externos, como o número de CPUs;
3. Nenhuma tarefa fora da região crítica pode bloquear qualquer processo;
4. Processos não podem esperar para sempre (starvation)



Algoritmos para possibilitar o acesso compartilhado

Espera ocupada

Variável **Turn** indica de quem é a vez de entrar na região crítica

```
while (TRUE) {  
    while (turn != 0) { // ;}  
    critical_region( );  
    turn = 1;  
    noncritical_region( );  
}
```

(a)

Processo A

Enquanto não é a sua vez, fica verificando de pode entrar

```
while (TRUE) {  
    while (turn != 1) { // ;}  
    critical_region( );  
    turn = 0;  
    noncritical_region( );  
}
```

(b)

Processo A

Espera ocupada

Se a fica pra sempre aqui, o Processo B nunca consegue acessar a região crítica!

```
while (TRUE) {  
    while (turn != 0) ;  
    critical_region( );  
    turn = 1;  
    noncritical_region( );  
}
```

(a)

Processo A

STARVATION!

```
while (TRUE) {  
    while (turn != 1) { // ;  
    critical_region( );  
    turn = 0;  
    noncritical_region( );  
}
```

(b)

Processo A

Algoritmo de Peterson



Algoritmo de Peterson

```
int turno;
bool interesse[2] = {false, false};

// Processo 0
void P0() {
    while (true) {
        interesse[0] = true; turno = 1;
        while (interesse[1] && turno == 1);
        printf("P0 na região crítica\n");
        interesse[0] = false;
    }
}
```

```
// Processo 1
void P1() {
    while (true) {
        interesse[1] = true; turno = 0;
        while (interesse[0] && turno == 0);
        printf("P1 na região crítica\n");
        interesse[1] = false;
    }
}
```

Algoritmo de Peterson

```
int turno;
bool interesse[2] = {false, false};

// Processo 0
void P0() {
    while (true) {
        interesse[0] = true; turno = 1;
        while (interesse[1] && turno == 1);
        printf("P0 na região crítica\n");
        interesse[0] = false;
    }
}
```

Evita Starvation, mas ainda utiliza
espera ocupada! Desperdiçando
recursos!

```
// Processo 1
void P1() {
    while (true) {
        interesse[1] = true; turno = 0;
        while (interesse[0] && turno == 0);
        printf("P1 na região crítica\n");
        interesse[1] = false;
    }
}
```

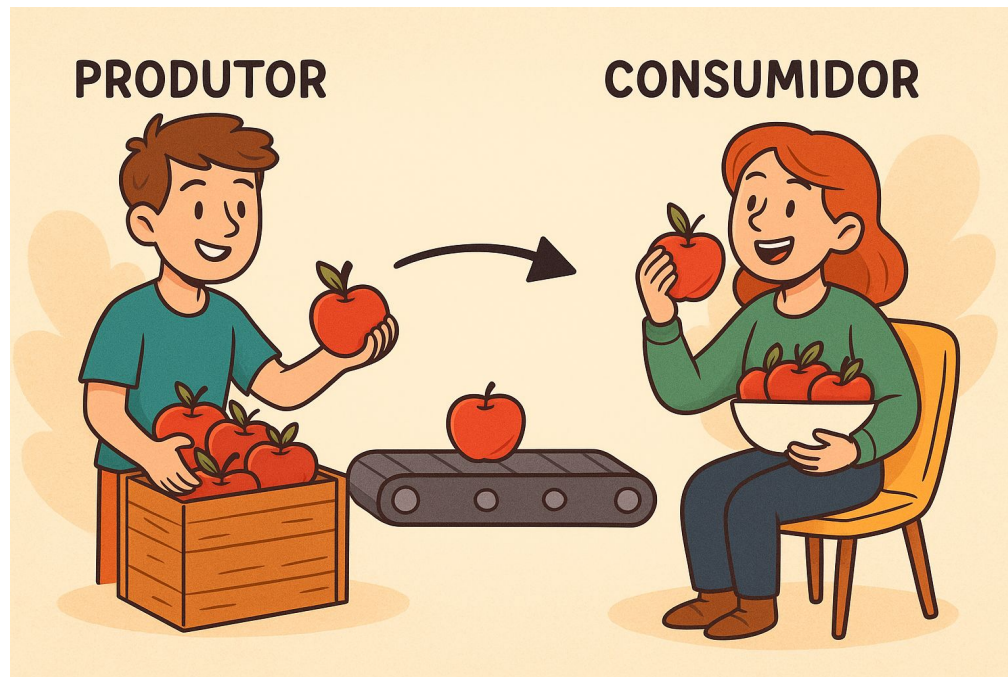
Dormir e acordar

Ao invés de ficar esperando, os processos podem **agora enviar sinais** um para o outro:

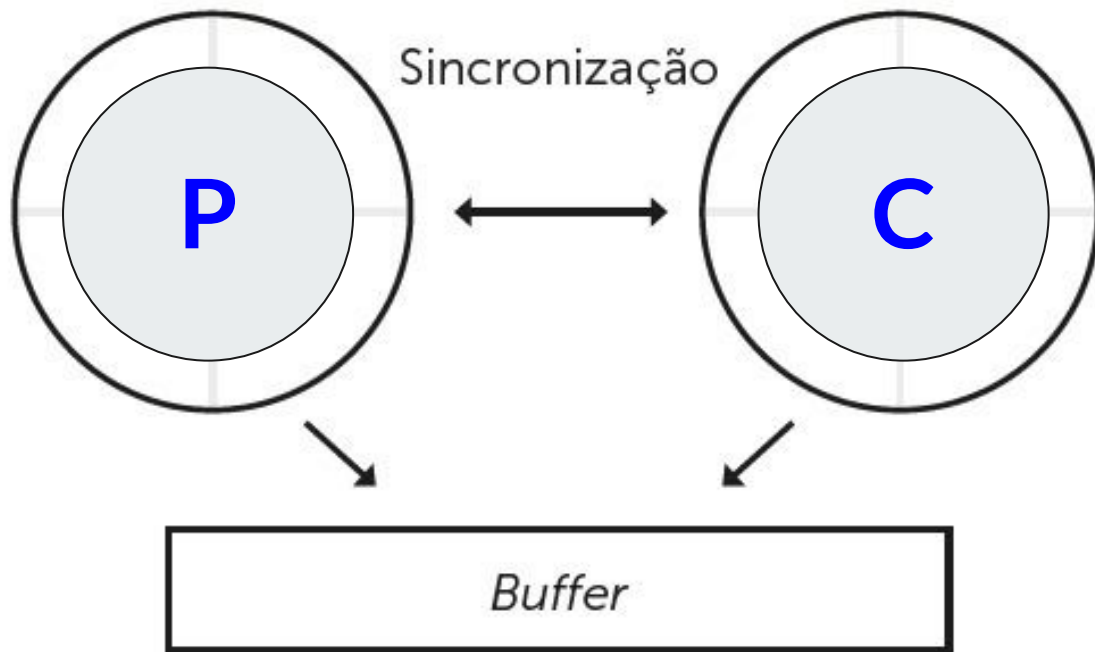
- **Sleep():** Se coloca para dormir caso a região crítica esteja ocupada (economizando recursos);
- **Wakeup():** Acorda o outro processo que está dormindo;



Problema do Produtor-Consumidor



Problema do Produtor-Consumidor



Problema do Produtor-Consumidor

```
#define N 100
int count = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        if (count == N) sleep();
        insert_item(item);
        count = count + 1;
        if (count == 1) wakeup(consumer);
    }
}
```

Produtor

```
void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0) sleep();
        item = remove_item();
        count = count - 1;
        if (count == N - 1) wakeup(producer);
        consume_item(item);
    }
}
```

Consumidor

Problema do Produtor-Consumidor

```
#define N 100  
int count = 0;
```

```
void producer(void)
```

```
{
```

```
    int item;
```

```
    while (TRUE) {
```

```
        item = produce_item();
```

```
        if (count == N) sleep();
```

```
        insert_item(item);
```

```
        count = count + 1;
```

```
        if (count == 1) wakeup(consumer);
```

```
    }
```

```
}
```

Produtor

Se o produtor envia o sinal de Wakeup(), mas o consumidor ainda não dormiu, o sinal de wakeup() é perdido. Os dois processos dormirão pra sempre.

Não garante exclusão mútua

```
void consumer(void)
```

```
{
```

```
    int item;
```

```
    while (TRUE) {
```

```
        if (count == 0) sleep();
```

```
        item = remove_item();
```

```
        count = count - 1;
```

```
        if (count == N - 1) wakeup(producer);
```

```
        consume_item(item);
```

```
    }
```

```
}
```

Consumidor

Problema do Produtor-Consumidor

```
#define N 100  
int count = 0;
```

DEADLOCK!

```
void producer(void)  
{  
    int item;  
  
    while (TRUE) {  
        item = produce_item();  
        if (count == N) sleep();  
        insert_item(item);  
        count = count + 1;  
        if (count == 1) wakeup(consumer);  
    }  
}
```

Produtor

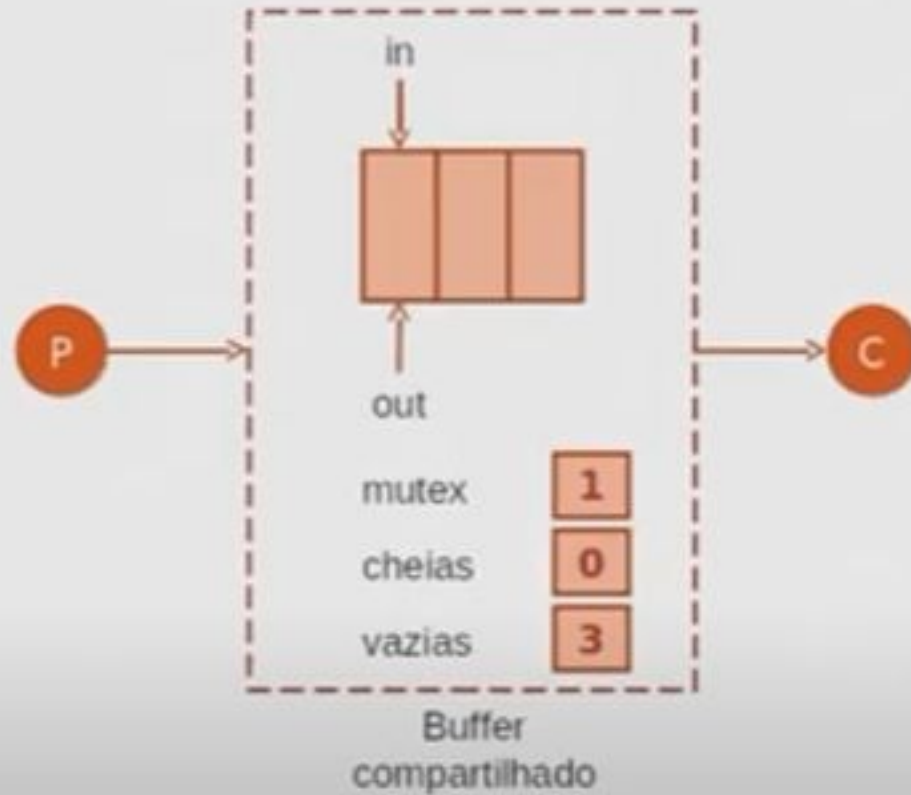
```
void consumer(void)  
{  
    int item;  
  
    while (TRUE) {  
        if (count == 0) sleep();  
        item = remove_item();  
        count = count - 1;  
        if (count == N - 1) wakeup(producer);  
        consume_item(item);  
    }  
}
```

Consumidor

Semáforo



Semáforo



Semáforo

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;
```

Existem 3 semáforos:
mutex(controla a entrada na região crítica),
empty(quantidade de espaços vazios) e
full(quantidade de espaços cheios).

```
void producer(void)
```

```
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}
```

```
void consumer(void)
```

```
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

Os semáforos resolvem os problemas anteriores, garantindo a exclusão mútua. Porém devem ser bem implementados

Semáforo

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;
```

```
void producer(void)
```

```
{
```

```
    int item;
```

```
    while (TRUE) {
```

```
        item = produce_item();
```

```
        down(&empty);
```

```
        down(&mutex);
```

```
        insert_item(item);
```

```
        up(&mutex);
```

```
        up(&full);
```

```
    }
```

```
}
```

```
void consumer(void)
```

```
{
```

```
    int item;
```

```
    while (TRUE) {
```

```
        down(&full);
```

```
        down(&mutex);
```

```
        item = remove_item();
```

```
        up(&mutex);
```

```
        up(&empty);
```

```
        consume_item(item);
```

```
    }
```

```
}
```

Se for programado errado, e a ordem for trocado, por exemplo, os dois processos pode dormir infinitamente

DEADLOCK!

Monitor

- Uma proposta de mais alto nível;
- Implementado pelas linguagens de programação;
- **As linguagens são responsáveis por garantir a exclusão mútua;**
- A linguagem Java, por exemplo, utiliza a estrutura *Synchronized* com *Wait()*, *Notify()*, e *NotifyAll()*;

Na aula de Hoje, aprendemos:

- A importância da comunicação entre processos;
- Os conceitos de condição de corrida, área crítica e exclusão mútua;
- Espera ocupada, algoritmo de Peterson, Sleep() e Wakeup(), Produtor-consumidor, Semáforos e monitores;

Referencial Bibliográfico

- Maziero, Carlos Alberto – ***Sistemas Operacionais: Conceitos e Mecanismos***
- Tanenbaum, Andrew S.; Bos, Herbert – **Sistemas Operacionais Modernos** (tradução de Modern Operating Systems, 4ª edição);
- Silberschatz, Abraham; Galvin, Peter Baer; Gagne, Greg – ***Fundamentos de Sistemas Operacionais*** (tradução de *Operating System Concepts*, 9ª edição)

SISTEMAS OPERACIONAIS

Exit